

CS Study

Database - Database query

목차

1

DB 쿼리 관련 (SQL 실행 순서)

2

WHERE vs HAVING

3

GROUP BY

4

EXISTS vs IN

5

LIKE

목차

1

DB 쿼리 관련 (SQL 실행 순서)

2

WHERE vs HAVING

3

GROUP BY

4

EXISTS vs IN

5

LIKE

DB 쿼리란?

데이터베이스에 원하는 정보를 얻기 위해
특정 언어 (주로 SQL)로 작성된 **질문**



DB 쿼리 종류

- **DDL**
 - DB **구조 생성, 수정, 삭제** 명령어
- **DML**
 - DB 안 **실제 데이터 다루는** 명령어
- **DCL**
 - **접근 권한 제어**하는 명령어
- **TCL**
 - **트랜잭션 관련 관리 및 제어** 명령어



DML

- **SELECT**
 - 테이블에서 원하는 **데이터 조회 및 검색**
- **INSERT**
 - 테이블에 **새로운 데이터 추가**
- **UPDATE**
 - 테이블의 **기존 데이터 수정**
- **DELETE**
 - 테이블에서 **데이터 삭제**



SELECT문 형식

```
SELECT [PREDICATE] [테이블명.] 속성명 [AS 별칭] [, [테이블명.] 속성명, ...]  
      [, 그룹 함수(속성명) [AS 별칭]]  
      [, WINDOW 함수 OVER (PARTITION BY 속성명1, 속성명2, ...)]  
FROM 테이블명[, 테이블명, ... , [JOIN 테이블명]]  
[WHERE 조건1 [조건 연산자 조건2 ...]]  
[GROUP BY 속성명1, 속성명2, ...]  
[HAVING 조건]  
[ORDER BY 속성명 [ASC | DESC]];
```

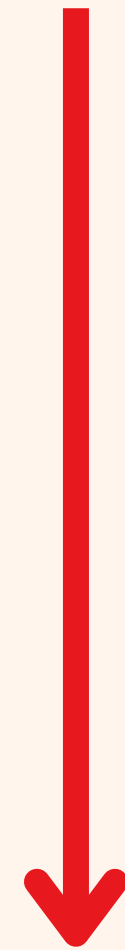
SQL 쿼리 실행 순서

FROM, ON, JOIN ⇒ WHERE

⇒ GROUP BY ⇒ HAVING

⇒ SELECT ⇒ DISTINCT

⇒ ORDER BY ⇒ LIMIT, OFFSET



```
SELECT DISTINCT department_id,  
                COUNT(*) AS employee_count  
FROM employees  
WHERE salary > 3000  
GROUP BY department_id  
HAVING COUNT(*) > 5  
ORDER BY employee_count DESC  
LIMIT 10;
```

SQL 쿼리 실행 순서

why 작성 순서 != 실행 순서??

- 논리적인 데이터 처리 흐름
 - 실행 순서가 **논리적으로 자연스러운** 순서
- 최적화 효율성
 - WHERE 같은 **필터 적용해 데이터 줄임**
 - **이후 단계 처리량 줄어** 성능 향상



목차

1

DB 쿼리 관련 (SQL 실행 순서)

2

WHERE vs HAVING

3

GROUP BY

4

EXISTS vs IN

5

LIKE

WHERE vs HAVING

WHERE

- 그룹화 전 필터링
- 행 단위 조건



HAVING

- 그룹화 후 필터링
- 그룹화 결과에 조건

WHERE vs HAVING

WHERE만 사용해 조건 걸 때

- 조건에 **필터링된 데이터만 다음 단계**로 넘어감
- **불필요한** 데이터 처리량 **사전 제거 가능**
- **INDEX** 적극적으로 **사용 가능**

```
SELECT *  
FROM employees  
WHERE salary > 3000;
```

WHERE vs HAVING

HAVING만 사용해 조건 걸 때

- 모든 데이터 먼저 그룹핑, 집계
- 집계 결과에 조건을 검
- 필요 없는 데이터까지 그룹핑, 연산
- INDEX 활용 못하는 상황 생김

```
SELECT COUNT(*) AS total_count  
FROM employees  
HAVING COUNT(*) > 10;
```

WHERE vs HAVING

성능 향상 방법

- **행 단위, 원본 테이블 관한 조건** 항상 WHERE절
- WHERE절로 거를 수 있으면 **우선 필터링**
- **WHERE절에 올 수 있는 조건** HAVING에 X
- **집계값, 그룹 결과 관한 조건만** HAVING에



목차

1

DB 쿼리 관련 (SQL 실행 순서)

2

WHERE vs HAVING

3

GROUP BY

4

EXISTS vs IN

5

LIKE

GROUP BY

GROUP BY란?

속성 기준으로 데이터 묶어 그룹화 해 검색할 때 사용

같은 값 갖는 속성들 하나의 그룹으로 묶음

성능에 영향 가능해 적절한 인덱스 사용과 주의 필요



GROUP BY

성능 영향 주는 이유??

- 대량의 데이터 **정렬 및 그룹화**
- INDEX 미활용 시 **전체 테이블 스캔**
- 임시 테이블과 **디스크 IO** 과다
- 복잡한 집계 함수, 조인 시 **INDEX 무효화**



GROUP BY

대량의 데이터 정렬 및 그룹화

컬럼 기준으로 **데이터 그룹화, 정렬**

데이터 많으면 작업 무거워짐

그러면 **자원 많이 사용**

INDEX 미활용 시 전체 테이블 스캔

행들 **그룹화위해 행들 읽어야 함**

대상 컬럼 **INDEX 없으면 전체 스캔**

그 후 **임시 테이블에 담아** 그룹화 처리

GROUP BY

임시 테이블과 디스크 IO 과다

복잡한 함수, 조인 시 INDEX 무효화

메모리에 임시 테이블 생성

GROUP BY에 함수 있으면 INDEX 무효화

메모리 한계 초과 시 디스크에 생성

INDEX는 원본 값 기반으로 정렬

디스크 IO로 쿼리 매우 느려질 수 있음

함수 적용 후 값 달라져 INDEX 무효화

GROUP BY

성능 개선 방법

- INDEX 활용 최적화

- GROUP BY 컬럼에 INDEX
- 컬럼 여러 개인 경우 모두 INDEX 생성

- 컬럼 관련 개선

- GROUP BY 컬럼, 순서 조정
- 함수나 계산식 GROUP BY에 X

- 쿼리 구조 개선

- GROUP BY 전에 WHERE로 데이터 최대한 필터링
- JOIN전에 서브 쿼리 등으로 미리 집계

목차

1

DB 쿼리 관련 (SQL 실행 순서)

2

WHERE vs HAVING

3

GROUP BY

4

EXISTS vs IN

5

LIKE

EXISTS vs IN

구분	EXISTS	IN
용도	서브쿼리 결과의 존재 유무 판단 주 테이블 값과 비교해 서브쿼리에 포함 여부 판단 시 사용	특정 값이 집합/서브쿼리 결과에 포함 되는지 확인
동작 방식	메인 쿼리의 각 행마다 서브쿼리 실행, 조건 만족하면 TRUE, 즉시 평가 종료(쇼트서킷)	IN 리스트 전체(서브쿼리 결과 포함) 에 대해 값을 비교
NULL 처리	NULL 값이 있어도 정상적으로 처리 조건만 맞으면 TRUE	NULL이 포함된 경우(특히 NOT IN) 올바르지 않은 결과 가능
사용 예	WHERE EXISTS (SELECT ...)	WHERE id IN (SELECT ...)
비교 대상	값의 존재만 체크(값 반환 X)	동일 값 집합 내 존재 여부 직접 비교

EXISTS vs IN

IN절에 NULL 있는 경우 동작 방식

- 컬럼의 값이 값 list or 서브쿼리 **결과에 포함되면 TRUE**
- IN절 **내부적으로 OR 연산자로 처리**
- NULL은 “값이 없음” 의미해 어떤 **값과 비교 불가능**
- NULL 값 가진 행은 **FALSE로 취급**
- **NULL 값 나온 행이 조회 X**

```
WHERE column IN (1, 2, NULL)
```

```
WHERE column = 1 OR column = 2 OR column = NULL
```

EXISTS vs IN

NOT IN절에 NULL 있는 경우 동작 방식

- 컬럼의 값이 값 list or 서브쿼리 **결과에 하나도 포함 X $\Rightarrow$ TRUE**
- NOT IN절 **내부적으로 AND 연산자로 처리**
- **NULL과 비교한 것과 AND해 결과 모두 UNKNOWN 평가됨**
- **아무 행도 반환X 문제 발생**

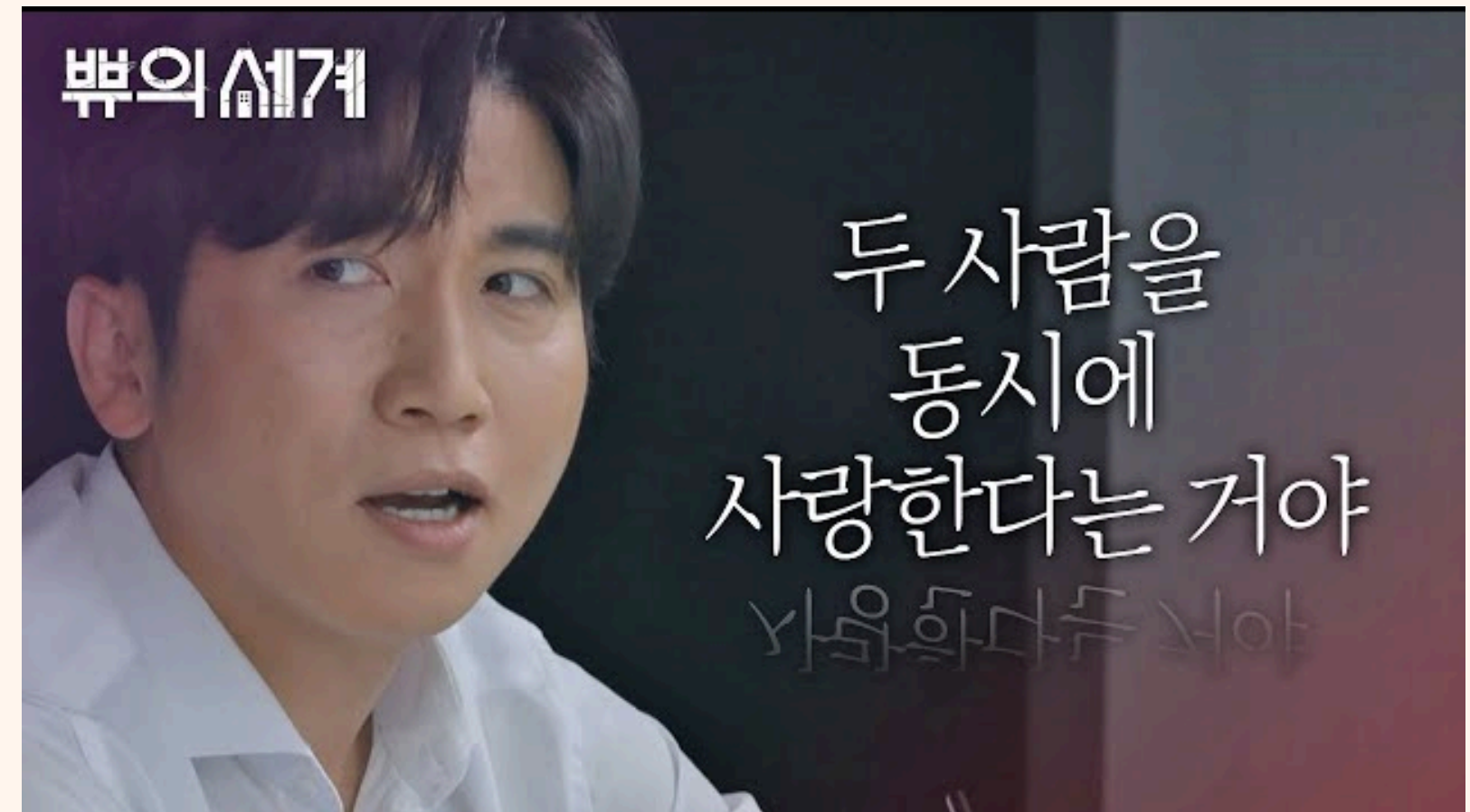
```
WHERE column NOT IN (1, 2, NULL)
```

```
WHERE column != 1 AND column != 2 AND column != NULL
```

EXISTS vs IN

EXISTS, IN 성능 관련

- 서브 쿼리 결과 적을 때
 - $IN > EXISTS$
- 서브 쿼리 결과 많을 때
 - $IN < EXISTS$
- 하나의 값만 검사할 때
 - $IN > EXISTS$
- 서브 쿼리 컬럼 INDEX 있을 때
 - $IN < EXISTS$



EXISTS vs IN

서브 쿼리 결과 적을 때

- **IN > EXISTS**
- IN절 결과 **메모리 상에 미리 만듦**
- 만든 결과와 **컬럼 값 OR**
- EXISTS는 **각 행마다 서브 쿼리 실행**
- **서브 쿼리 반복 실행**으로 느림

서브 쿼리 결과 많을 때

- **IN < EXISTS**
- IN은 **전체 결과 모아서 비교**해서 느림
- EXISTS는 **처음 TRUE되는 순간** **검색 멈춰** 빠름

EXISTS vs IN

하나의 값만 검사

- **IN > EXISTS**

- 대부분 '**= 연산자**'와 동일하게 최적화
- 성능 이슈 x
- 단순 명료
- IN이 **간단하고 자연스럽게 읽힘**

서브 쿼리 컬럼 INDEX 있을 때

- **IN < EXISTS**

- **서브 쿼리 컬럼**에 INDEX
- 조건에 부합하는 결과 **서브 쿼리 결과**에서 **INDEX로 빠르게 찾을**
- INDEX 없다면 **FULL TABLE SCAN**

EXISTS vs IN

```
SELECT *  
FROM users u  
WHERE EXISTS (  
    SELECT 1  
    FROM orders o  
    WHERE o.user_id = u.id AND o.status = 'pending'  
);
```

- 원래는 users의 **u_id 컬럼 값이 1001**이면
- 서브 쿼리의 **user_id 컬럼 값 1001인것**
full table scan
- 서브 쿼리의 **user_id 컬럼**에 index 있다면
빠르게 찾아서
- **서브 쿼리 결과에 exists** 한다고 판단

목차

1

DB 쿼리 관련 (SQL 실행 순서)

2

WHERE vs HAVING

3

GROUP BY

4

EXISTS vs IN

5

LIKE

LIKE

LIKE 연산자

SELECT, UPDATE, DELETE
등에서 **문자열 패턴 매칭에 사용**



LIKE

LIKE 패턴 검색이 성능 문제 일으키는 이유

- INDEX 활용 못해 **전체 테이블 스캔 발생**
- **복잡한 문자 비교** (인코딩, 대소문자..)
- 데이터 양 많고 **패턴 복잡**한 경우

LIKE '%pattern'

➡ 이런 경우에 성능 문제!

LIKE

LIKE '%pattern'

- 패턴 앞부분에 % 있는 경우
- %는 0개 이상의 임의 문자를 의미
- DB는 해당 컬럼의 모든 값 다 확인 필요
- INDEX 활용 못해 전체 테이블 스캔 발생



LIKE

복잡한 문자 비교

- 단순한 문자 비교 X
- 문자 인코딩, 대소문자 등 **복합 작용**
- **비교 연산 비용 증가**

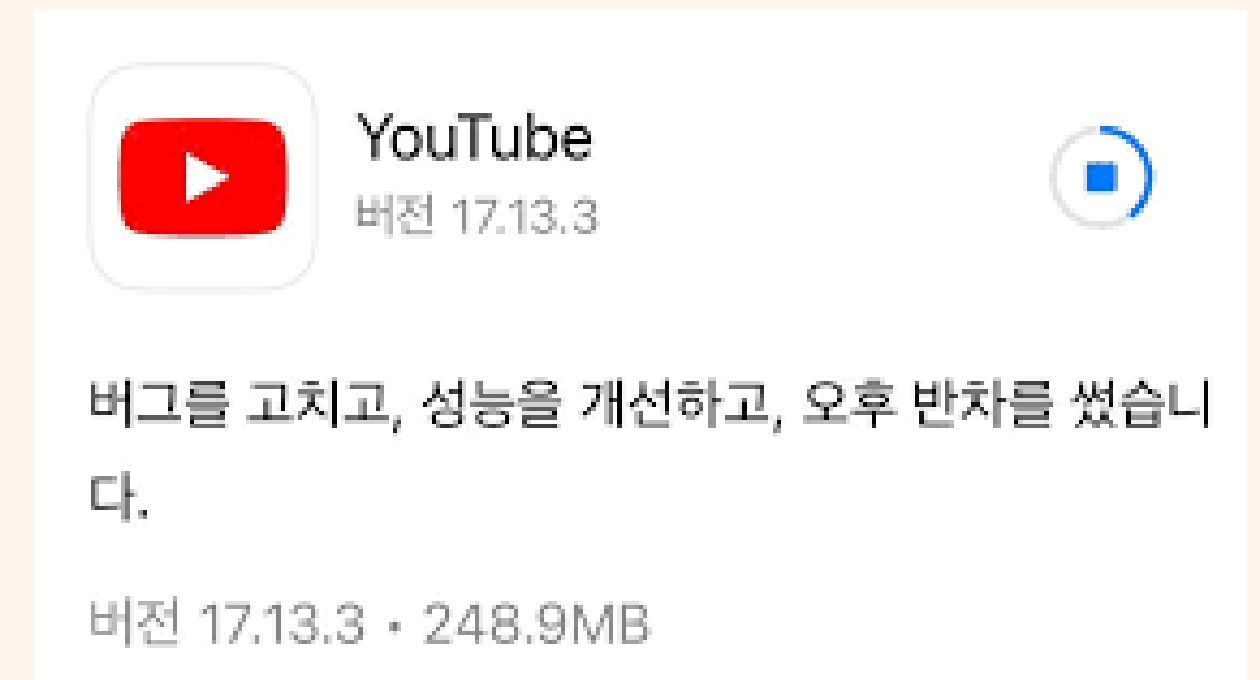
데이터 양과 패턴 복잡도

- 패턴 검색할 테이블 **데이터 양 많은 경우**
- **패턴이 매우 복잡**한 경우
- 이때 **자주 패턴 검색 실행되면 성능 저하**

LIKE

성능 개선 방법

- **패턴 앞부분에 %** 피하기
 - LIKE 'abc%' 처럼 고정 prefix 검색으로 INDEX 사용
- **Full-Text INDEX** or **전문 검색 엔진** 활용
 - LIKE '%pattern' 형태 반드시 필요한 경우
- REGEXP_LIKE 등 더 **복잡한 패턴 처리 문법 활용**
 - 복잡한 패턴 검색 가능하게 해주는 정규식 기반 패턴 매칭 함수



Quiz

1번 문제

쿼리의 실행 순서는?

1번 문제 정답

FROM, ON, JOIN ➡ WHERE ➡ GROUP BY ➡ HAVING

➡ SELECT ➡ DISTINCT ➡ ORDER BY ➡ LIMIT, OFFSET

2번 문제

GROUP BY가 성능에 영향주는 이유는??

2번 문제 정답

GROUP BY가 성능에 영향 주는 이유??

- 대량의 데이터 **정렬 및 그룹화** 수행
- 행들 **그룹화** 해야 해서 읽어야 하는데 INDEX 미활용 시 **전체 테이블 스캔**
- 임시 테이블 **메모리 한계 초과**해 디스크에 생기면서 **디스크 IO** 과다
- 함수, 계산식 사용 시 **값 바뀌어 원본 값 기반 정렬된 INDEX** 무효화

3번 문제

LIKE 패턴 검색이 성능 문제 일으키는 이유는??

3번 문제 정답

LIKE 패턴 검색이 성능 문제 일으키는 이유

- 패턴 앞부분에 % 있는 경우 문제 발생
- %라 모든 값을 다 확인해야 함
- INDEX 활용 못해 전체 테이블 스캔 발생